

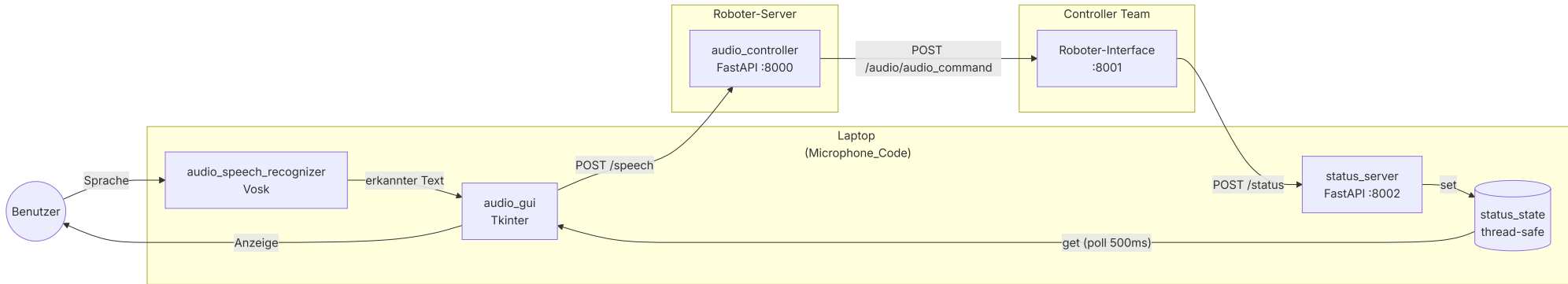
API-Verlauf

Erklärung der Schnittstellen und des Datenflusses zwischen Audio-Team und Controller.

Beteiligte Komponenten

- **Microphone_Code/** — Client auf dem Laptop (GUI, Spracherkennung, Status-Empfänger)
- **Roboter_Code/** — FastAPI-Server für Sprachbefehle (Befehlserkennung + Weiterleitung)
- **Shared_Code/** — Gemeinsamer Parser, von beiden Seiten importiert
- **Roboter-Interface** — Externe Komponente eines anderen Teams unter 10.144.65.64:8001

1. Gesamtarchitektur

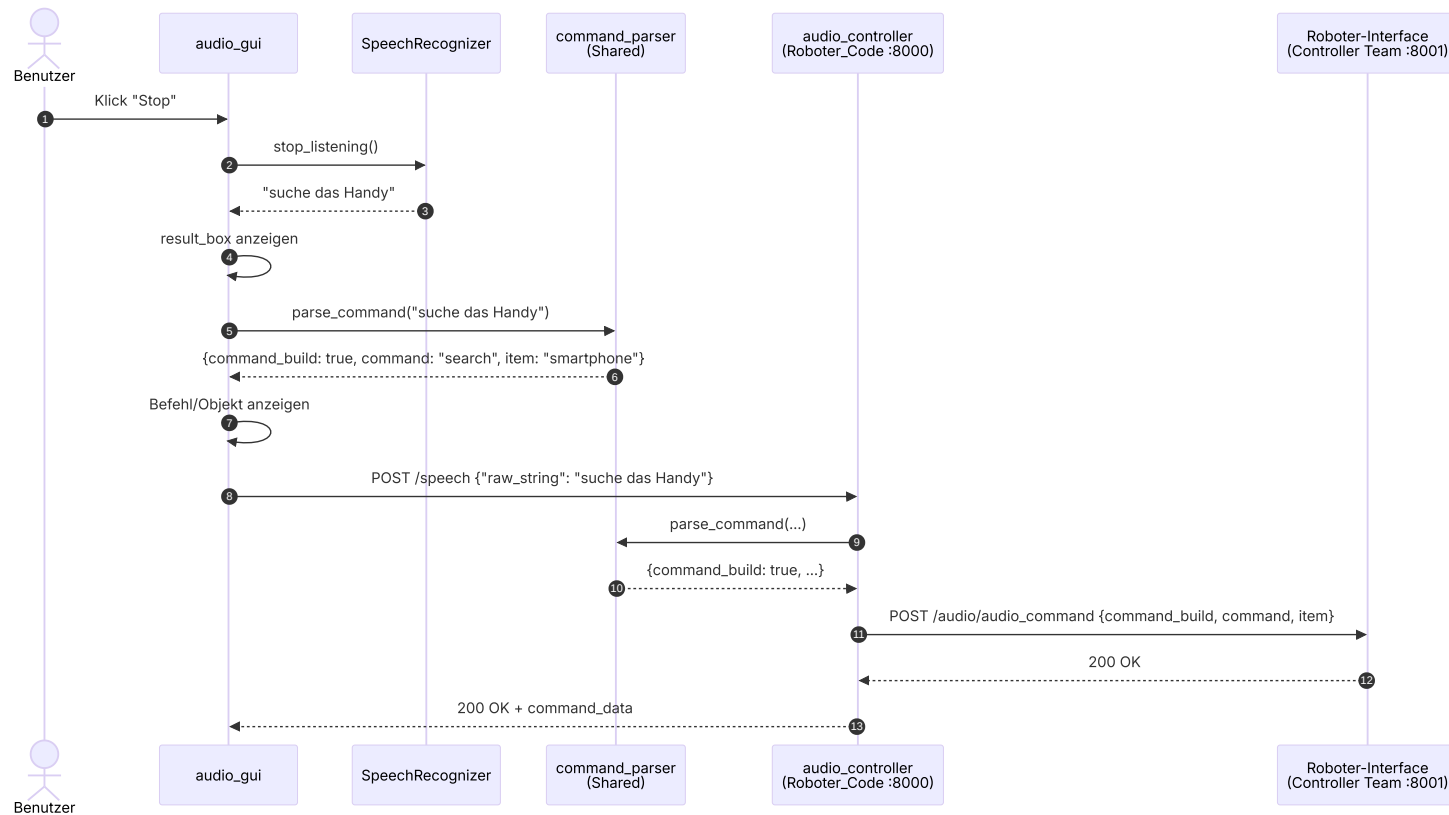


2. Übersicht aller Endpunkte

Endpunkt	Host:Port	Methode	Wer ruft auf	Wer hostet	Zweck
/	127.0.0.1:8000	GET	(Health-Check)	audio_controller	Lebenszeichen
/speech	127.0.0.1:8000	POST	audio_gui	audio_controller	Sprachbefehl absetzen
/audio/audio_command	10.144.65.64:8001	POST	audio_controller	Controller Team	Befehl an Roboter weiterleiten
/status	<laptop>:8002	POST	Controller Team	status_server	Roboter meldet Statuswechsel

3. Vorwärts-Pfad: Sprachbefehl → Roboter

Ablauf vom Drücken des **Stop**-Buttons bis zur Aktion des Roboters.

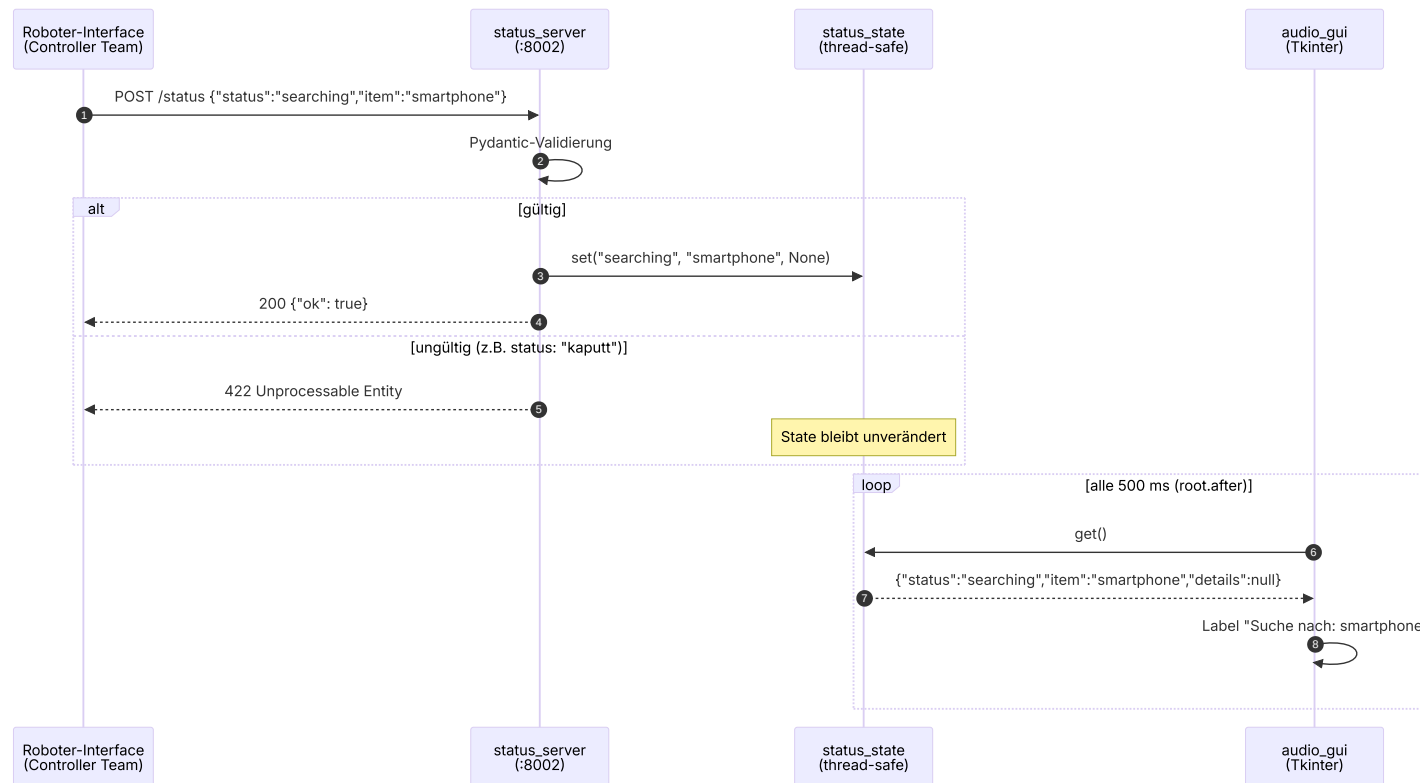


🔗 Warum wird **zweimal** geparkt (im Client und im Server)?

Der Client parst lokal, damit die GUI auch dann Befehl und Objekt anzeigen kann, wenn der Server nicht erreichbar ist. Der Server parst zusätzlich, weil er die einzige Stelle ist, die das Roboter-Interface ansprechen darf. Der gemeinsame [command_parser](#) in `Shared_Code/` stellt sicher, dass beide Seiten dasselbe Ergebnis liefern.

4. Rückwärts-Pfad: Roboter-Status → GUI

Sobald der Roboter etwas tut (Suche beginnen, Ziel finden, Fehler), schickt der externe Controller einen POST an den Laptop.



🔗 Warum Polling und nicht direktes Callback?

Tkinter ist single-threaded. Aus einem fremden Thread (hier: dem FastAPI-Server-Thread) darf man Tkinter-Widgets nicht direkt anfassen. Lösung: Der Server schreibt in den thread-sicheren [status_state](#), und der Tkinter-Hauptthread liest periodisch mit `root.after(500, ...)`.

5. Datenmodelle

raw_string (POST /speech)

```
{ "raw_string": "suche das Handy" }
```

command_data (Antwort von /speech und Body an Roboter-Interface)

```
{
  "command_build": true,
  "command": "search",
  "item": "smartphone"
}
```

Bei nicht erkanntem Befehl:

```
{ "command_build": false }
```

StatusUpdate (POST /status, Pydantic-validiert)

```
{
  "status": "searching",

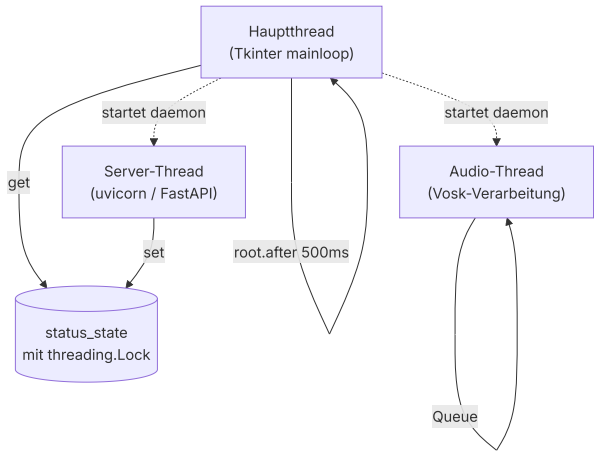
```

```
"item": "smartphone",
"details": null
}
```

status muss einer der folgenden Enum-Werte sein (nur Vorschlag - können noch geändert werden):

Wert	Bedeutung	Anzeige in GUI
idle	Bereit / nichts zu tun	"Bereit"
searching	Sucht aktiv nach Item	"Suche nach: <item>"
found	Item gefunden	"Ziel gefunden: <item>"
lost	Item verloren	"Ziel verloren"
error	Fehlerzustand	"Fehler: <details>"

6. Threading-Modell



⚠ Wichtig

Alle Hilfs-Threads laufen als **daemon=True**. Dadurch sterben sie automatisch, wenn das Tkinter-Fenster geschlossen wird — kein manuelles Aufräumen, keine Zombie-Prozesse.

7. Fehlerfälle und Verhalten

Situation	Beobachtetes Verhalten
Roboter-Server (:8000) nicht erreichbar	GUI zeigt Befehl/Objekt trotzdem; rote Zeile "Server nicht erreichbar"
Kein bekannter Befehl im Text	Befehl/Objekt = - ; rote Zeile "Kein Befehl erkannt"
/status mit ungültigem status	HTTP 422; State bleibt auf letztem gültigen Wert
Roboter-Interface (:8001) nicht erreichbar	Server loggt "Roboter-Interface nicht erreichbar!" — Client bekommt aber trotzdem command_data zurück
Externes Team sendet keine Status-Updates	GUI bleibt auf "Bereit"

8. Konfiguration und Ports

Komponente	Adresse	Quelle
Roboter-Server	127.0.0.1:8000/speech	Microphone_Code/audio_main.py (robot_url)

Komponente	Adresse	Quelle
Roboter-Interface	<IP-Roboter>:8001/audio/audio_command	Roboter_Code/audio_controller.py (interface_api)
Status-Server	0.0.0.0:8002/status	Microphone_Code/status_server.py (run(...))
Vosk-Modell	Roboter01.venv/vosk-model-small-de-0.15/	audio_speech_recognizer.py (MODEL_PATH)

9. Offene Klärungspunkte

🕒 Mit Roboter-Team abzustimmen

1. **IP/Hostname des Laptops** — wie kennt der Roboter unsere Adresse für POST /status ?
2. **Port 8002 für /status** — ok oder gewünschter anderer Port?
3. **Vollständigkeit der Status-Enum** — fehlen returning , paused o.ä.?
4. **Sprache von item** — englische COCO-Klassen (z.B. cell phone , dining table) oder deutsch?
5. **details -Feld** — Freitext oder definierte Fehlercodes?
6. **Update-Frequenz** — bei jedem Wechsel oder periodisch?
7. **Heartbeat/Timeout** — nach X Sekunden ohne Update auf "Verbindung verloren" gehen?
8. **Authentifizierung** — Token im Header oder offen im LAN?

10. Komponenten-Referenz

audio_gui

Datei: Microphone_Code/audio_gui.py

Rolle: Tkinter-Oberfläche, schickt /speech , liest [status_state](#) für Anzeige.

audio_controller

Datei: Roboter_Code/audio_controller.py

Rolle: FastAPI-Server (:8000), nimmt /speech entgegen, parst, leitet an Roboter-Interface weiter.

status_server

Datei: Microphone_Code/status_server.py

Rolle: FastAPI-Server (:8002) auf dem Laptop, nimmt /status -Pushes vom Controller entgegen und schreibt sie in [status_state](#).

status_state

Datei: Microphone_Code/status_state.py

Rolle: Thread-sicherer Container — Brücke zwischen FastAPI-Thread (Schreiber) und Tkinter-Thread (Leser).

command_parser

Datei: Shared_Code/command_parser.py

Rolle: Übersetzt deutschen Sprachbefehl in {command, item} mit COCO-Klassennamen. Wird von beiden Seiten via sys.path -Injection importiert.

audio_speech_recognizer

Datei: Microphone_Code/audio_speech_recognizer.py

Rolle: Offline-Spracherkennung via Vosk in eigenem Daemon-Thread.

11. Schnellreferenz für Tests

```
# venv aktivieren
source Roboter01.venv/bin/activate

# Roboter-Server starten (Terminal 1)
cd Roboter_Code && uvicorn audio_controller:app --host 0.0.0.0 --port 8000

# Microphone-Client starten (Terminal 2)
cd Microphone_Code && python audio_main.py

# Status manuell pushen (Terminal 3)
```

```
curl -X POST http://127.0.0.1:8002/status \  
-H "Content-Type: application/json" \  
-d '{"status":"searching","item":"smartphone"}'  
  
# Sprachbefehl simulieren ohne Mikrofon  
curl -X POST http://127.0.0.1:8000/speech \  
-H "Content-Type: application/json" \  
-d '{"raw_string":"suche das Handy}'
```